

```

/**
 * server/services/resurrection-engine.ts
 *
 * Resurrection Engine – Nothing Stays Dead Forever
 *
 * Scans for "permanently failed" items across the entire pipeline and gives
 * them another chance after a cooldown window. No item is ever truly permanent.
 *
 * Philosophy:
 *   - YouTube changes formats, lifts restrictions, fixes API bugs
 *   - Keyword sanitizer gets fixed → old rejections should retry
 *   - FFmpeg encoding constraints get updated → old timeouts should retry
 *   - Quota resets daily → quota-blocked items retry next window
 *   - Network issues are transient → timeouts retry after cooldown
 *
 * Cooldown windows by failure reason:
 *   invalid_keywords    → 24h   (sanitizer fix should be deployed)
 *   format_unavailable → 72h   (YouTube sometimes restores formats)
 *   yt_dlp_timeout      → 6h    (transient throttling)
 *   quota_exceeded      → next  quota reset (07:05 UTC)
 *   ffmpeg_timeout      → 24h   (fresh boot, new constraints)
 *   ai_queue_full       → 2h    (saturation clears)
 *   network_timeout     → 4h    (transient)
 *   unknown             → 48h   (conservative default)
 *
 * Resurrection lifecycle:
 *   permanently_failed → pending (reset attempts, increment resurrection_count)
 *   If resurrection_count > MAX_RESURRECTIONS → extended_cooldown (7 days)
 *   If resurrection_count > EXTENDED_MAX → archived (human review needed)
 *
 * Runs every 4 hours on a jittered interval.
 */

```

```

import { createLogger }      from '../lib/logger.js';
import { setJitteredInterval } from '../lib/timer-utils.js';
import { storage }           from '../storage.js';
import { db }                from '../db.js';
import { sql }               from 'drizzle-orm';

```

```
const log = createLogger('resurrection-engine');
```

```
// ——— Config ———
```

```

const MAX_RESURRECTIONS    = 10;   // after 10 resurrections → extended cooldown
const EXTENDED_MAX         = 20;   // after 20 resurrections → archive for review
const RUN_INTERVAL_MS      = 4 * 60 * 60 * 1000; // every 4 hours

```

```
// YouTube quota resets at 07:05 UTC
```

```

function nextQuotaResetMs(): number {
    const now    = new Date();
    const reset = new Date(now);
    reset.setUTCHours(7, 5, 0, 0);
    if (reset <= now) reset.setUTCDate(reset.getUTCDate() + 1);
    return reset.getTime() - now.getTime();
}

// ——— Cooldown map —————

type FailureReason =
    | 'invalid_keywords'
    | 'format_unavailable'
    | 'yt_dlp_timeout'
    | 'quota_exceeded'
    | 'ffmpeg_timeout'
    | 'ai_queue_full'
    | 'network_timeout'
    | 'unknown';

function classifyError(errorMessage: string): FailureReason {
    const msg = errorMessage.toLowerCase();
    if (msg.includes('invalid video keywords') || msg.includes('invalid keyword'))
        return 'invalid_keywords';
    if (msg.includes('requested format is not available') || msg.includes('format_unavailable'))
        return 'format_unavailable';
    if (msg.includes('timed out') && msg.includes('yt-dlp'))
        return 'yt_dlp_timeout';
    if (msg.includes('quota') && (msg.includes('exceeded') || msg.includes('cap reached')))
        return 'quota_exceeded';
    if (msg.includes('ffmpeg') && msg.includes('timeout'))
        return 'ffmpeg_timeout';
    if (msg.includes('ai queue full'))
        return 'ai_queue_full';
    if (msg.includes('timeout') || msg.includes('aborted') || msg.includes('econnreset'))
        return 'network_timeout';
    return 'unknown';
}

function getCooldownMs(reason: FailureReason, resurrectionCount: number): number {
    // Escalate cooldown for repeated failures
    const multiplier = Math.min(resurrectionCount, 5); // cap at 5x

    const baseCooldowns: Record<FailureReason, number> = {
        invalid_keywords: 24 * 60 * 60 * 1000, // 24h
        format_unavailable: 72 * 60 * 60 * 1000, // 72h
        yt_dlp_timeout: 6 * 60 * 60 * 1000, // 6h
        quota_exceeded: nextQuotaResetMs(), // next reset window
        ffmpeg_timeout: 24 * 60 * 60 * 1000, // 24h
        ai_queue_full: 2 * 60 * 60 * 1000, // 2h
    };

```

```

network_timeout: 4 * 60 * 60 * 1000, // 4h
unknown:        48 * 60 * 60 * 1000, // 48h
};

const base = baseCooldowns[reason];
// Quota resets are always at a fixed time – don't multiply those
if (reason === 'quota_exceeded') return base;
return base * Math.max(1, multiplier);
}

```

// — Resurrection targets —

```

interface ResurrectionTarget {
  table:      string;
  idColumn:   string;
  statusColumn: string;
  errorColumn: string;
  retryAfterColumn: string;
  resurrectionCountColumn: string;
  attemptsColumn: string;
  failedStatuses: string[];
  resetStatus:  string;
  label:        string;
}

const TARGETS: ResurrectionTarget[] = [
  {
    table:      'youtube_push_backlog',
    idColumn:   'id',
    statusColumn: 'status',
    errorColumn: 'last_error',
    retryAfterColumn: 'retry_after',
    resurrectionCountColumn: 'resurrection_count',
    attemptsColumn: 'attempts',
    failedStatuses: ['permanently_failed', 'failed'],
    resetStatus: 'pending',
    label: 'YouTube push backlog',
  },
  {
    table:      'pre_encoder_queue',
    idColumn:   'id',
    statusColumn: 'status',
    errorColumn: 'last_error',
    retryAfterColumn: 'retry_after',
    resurrectionCountColumn: 'resurrection_count',
    attemptsColumn: 'attempt_count',
    failedStatuses: ['permanently_failed', 'failed'],
    resetStatus: 'pending',
    label: 'Pre-encoder queue',
  },
]

```

```

{
  table:                'shorts_clip_queue',
  idColumn:              'id',
  statusColumn:          'status',
  errorColumn:           'last_error',
  retryAfterColumn:      'retry_after',
  resurrectionCountColumn: 'resurrection_count',
  attemptsColumn:        'attempt_count',
  failedStatuses:        ['permanently_failed', 'failed'],
  resetStatus:            'pending',
  label:                  'Shorts clip queue',
},
{
  table:                'content_vault_backups',
  idColumn:              'id',
  statusColumn:          'status',
  errorColumn:           'error_message',
  retryAfterColumn:      'retry_after',
  resurrectionCountColumn: 'resurrection_count',
  attemptsColumn:        'download_attempts',
  failedStatuses:        ['permanently_failed', 'failed', 'download_failed'],
  resetStatus:            'pending',
  label:                  'Content vault downloads',
},
{
  table:                'video_metadata_sync_queue',
  idColumn:              'id',
  statusColumn:          'status',
  errorColumn:           'last_error',
  retryAfterColumn:      'retry_after',
  resurrectionCountColumn: 'resurrection_count',
  attemptsColumn:        'attempt_count',
  failedStatuses:        ['permanently_failed', 'failed'],
  resetStatus:            'pending',
  label:                  'Metadata sync queue',
},
];

// — Core resurrection logic —————

async function resurrectTarget(target: ResurrectionTarget): Promise<number> {
  const now = new Date();

  // Find items past their retry window
  const candidates = await db.execute(sql.raw(`
    SELECT
      ${target.idColumn}                AS id,
      COALESCE(${target.errorColumn}, '') AS error_message,
      COALESCE(${target.resurrectionCountColumn}, 0) AS resurrection_count,
      ${target.retryAfterColumn}          AS retry_after
  `));

```

```

FROM ${target.table}
WHERE ${target.statusColumn} = ANY(ARRAY[${target.failedStatuses.map(s =>
`${s}`)}.join(',')])])
AND (
    ${target.retryAfterColumn} IS NULL
    OR ${target.retryAfterColumn} <= NOW()
)
ORDER BY ${target.retryAfterColumn} ASC NULLS FIRST
LIMIT 100
`));

if (candidates.rows.length === 0) return 0;

let resurrected = 0;

for (const row of candidates.rows as any[]) {
    const resCount = Number(row.resurrection_count ?? 0);
    const errorMsg = String(row.error_message ?? '');
    const reason = classifyError(errorMsg);
    const itemId = row.id;

    // Archive if too many resurrections – but log it clearly
    if (resCount >= EXTENDED_MAX) {
        await db.execute(sql.raw(`
            UPDATE ${target.table}
            SET
                ${target.statusColumn} = 'archived',
                ${target.errorColumn} = ${target.errorColumn} || ' [Archived after ${EXTENDED_MAX}
resurrections – needs manual review]'
            WHERE ${target.idColumn} = '${itemId}'
        `));
        log.warn(
            `[Resurrection] ARCHIVED ${target.label} id=${itemId} after ${resCount} resurrections. `
+
            `Last error: ${errorMsg.slice(0, 100)}`
        );
        continue;
    }

    // Extended cooldown if above max but below extended max
    if (resCount >= MAX_RESURRECTIONS) {
        const extendedCooldown = 7 * 24 * 60 * 60 * 1000; // 7 days
        const retryAfter = new Date(Date.now() + extendedCooldown);
        await db.execute(sql.raw(`
            UPDATE ${target.table}
            SET
                ${target.statusColumn} = 'pending',
                ${target.attemptsColumn} = 0,
                ${target.retryAfterColumn} = '${retryAfter.toISOString()}',
                ${target.resurrectionCountColumn} = ${resCount + 1}

```

```

        WHERE ${target.idColumn} = '${itemId}'
    `));
    log.warn(
        `[Resurrection] Extended cooldown (7 days) for ${target.label} id=${itemId} ` +
        `(${resCount} prior resurrections). Reason: ${reason}`
    );
    resurrected++;
    continue;
}

// Normal resurrection
const cooldownMs = getCooldownMs(reason, resCount);
const retryAfter = new Date(Date.now() + cooldownMs);

await db.execute(sql.raw(`
    UPDATE ${target.table}
    SET
        ${target.statusColumn}          = '${target.resetStatus}',
        ${target.attemptsColumn}         = 0,
        ${target.retryAfterColumn}       = '${retryAfter.toISOString()}',
        ${target.resurrectionCountColumn} = ${resCount + 1}
    WHERE ${target.idColumn} = '${itemId}'
`));

const cooldownHours = Math.round(cooldownMs / 1000 / 60 / 60 * 10) / 10;
log.info(
    `[Resurrection] ✅ Resurrected ${target.label} id=${itemId} | ` +
    `reason=${reason} | resurrection #${resCount + 1} | ` +
    `retry in ${cooldownHours}h at ${retryAfter.toISOString()}`
);
resurrected++;
}

return resurrected;
}

// — Full resurrection cycle —————

export async function runResurrectionCycle(): Promise<void> {
    log.info('[Resurrection] Starting resurrection cycle');

    let totalResurrected = 0;

    for (const target of TARGETS) {
        try {
            const count = await resurrectTarget(target);
            if (count > 0) {
                log.info(`[Resurrection] ${target.label}: resurrected ${count} items`);
            }
            totalResurrected += count;
        }
    }
}

```

```

    } catch (err) {
      log.error(`[Resurrection] Error processing ${target.label}:`, err);
    }
  }

  if (totalResurrected > 0) {
    log.info(`[Resurrection] Cycle complete – ${totalResurrected} total items returned to
queue`);
  } else {
    log.info('[Resurrection] Cycle complete – nothing to resurrect');
  }
}

// — Service lifecycle —————

let stopResurrection: (() => void) | null = null;

export function startResurrectionEngine(): void {
  if (stopResurrection) {
    log.warn('[Resurrection] Already running');
    return;
  }

  log.info('[Resurrection] Starting – run interval: ~4 hours');

  // Run once 5 minutes after boot to catch anything that was waiting
  setTimeout(
    () => runResurrectionCycle().catch(err => log.error('[Resurrection] Startup run error:',
err)),
    5 * 60 * 1000
  );

  stopResurrection = setJitteredInterval(
    () => runResurrectionCycle().catch(err => log.error('[Resurrection] Cycle error:', err)),
    RUN_INTERVAL_MS,
  );
}

export function stopResurrectionEngine(): void {
  if (stopResurrection) {
    stopResurrection();
    stopResurrection = null;
    log.info('[Resurrection] Stopped');
  }
}

```